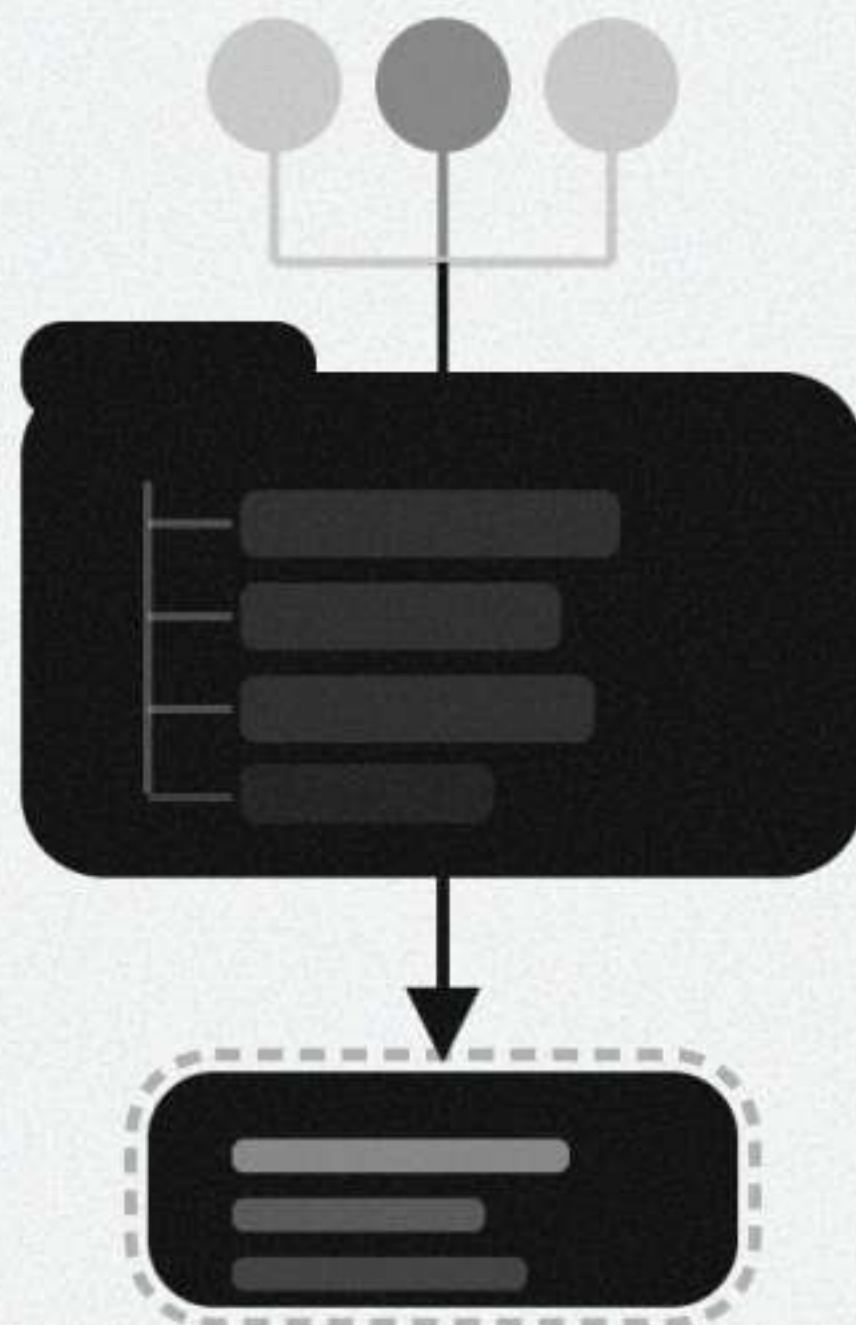


3 Ways **to understand** **ANY** **New Codebase**



SETUP / CONTEXT

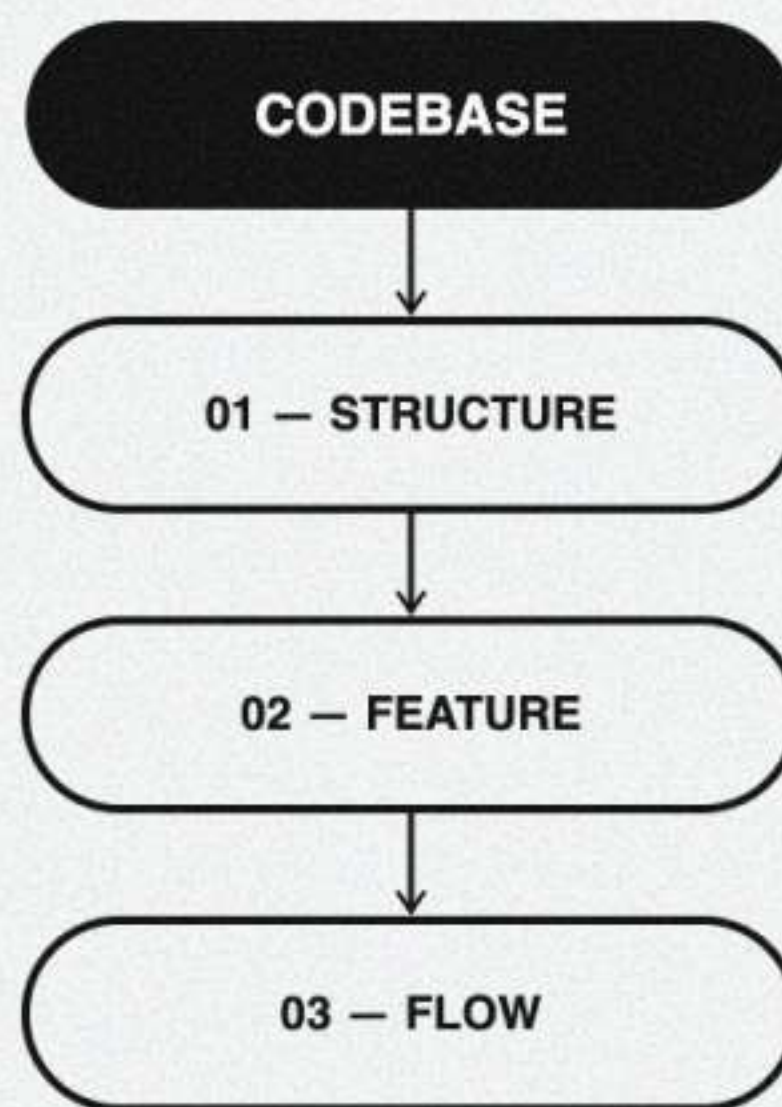
**Reading a new codebase can feel overwhelming.
You don't need to understand everything at once.**

Start with these 3 things:

01 — Map the structure

02 — Trace one feature

03 — Run it and follow the flow



you don't need to understand everything at once

1. Map the Structure

Don't start by opening random files.

First, figure out what each major folder is responsible for.

Look for:

- **Where does the UI live?**
- **Where are API calls made?**
- **Where is business logic?**
- **Where is shared code?**

src/

```
|— components/  
|— pages/  
|— hooks/  
|— lib/  
|— services/  
└— utils/
```



Clear structure

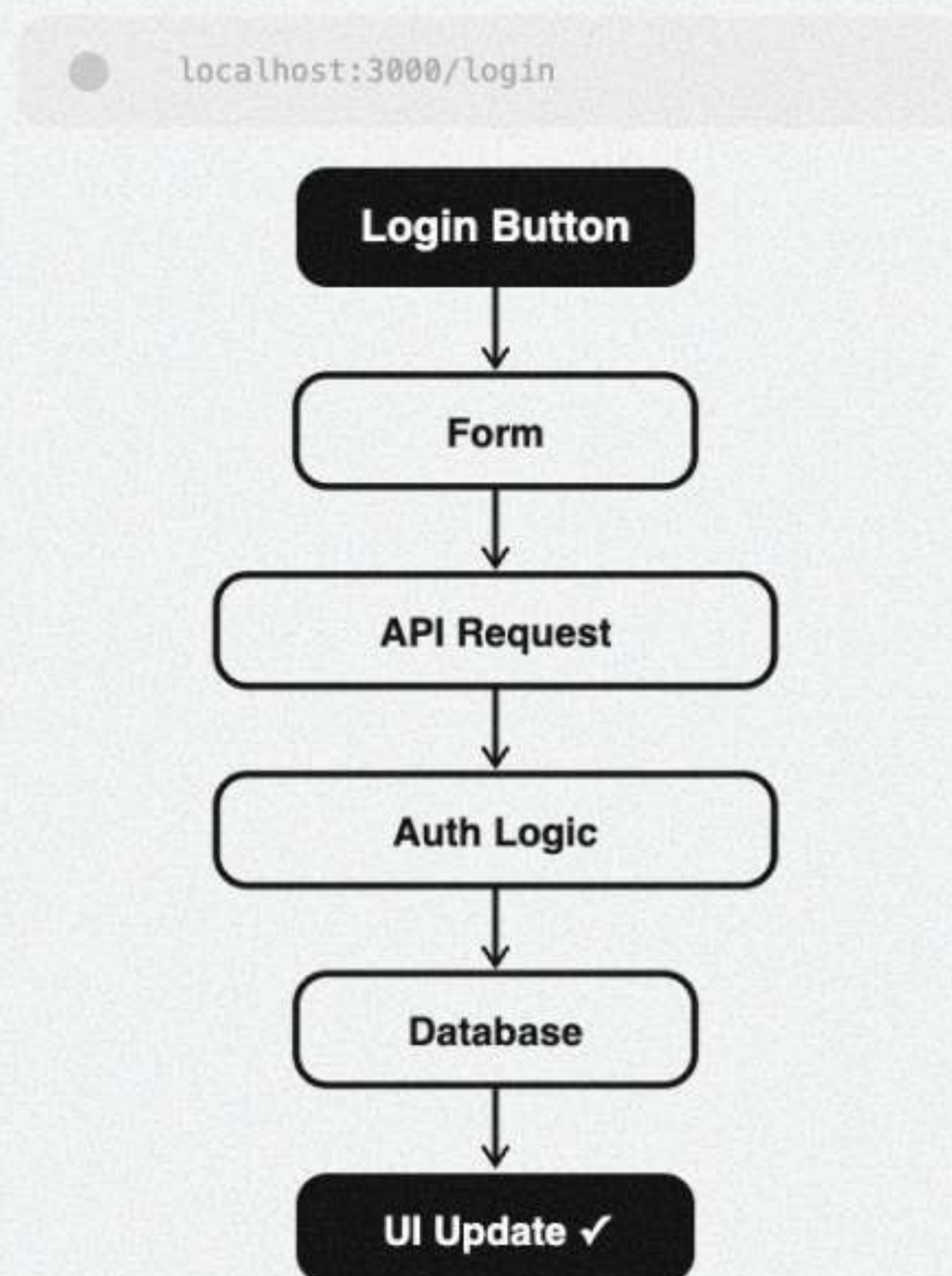


No context yet

2. Trace One Feature

Pick ONE thing the app does.
For example:
“What happens when I click
Login?”
Then follow it:

One feature can teach you how
the entire codebase connects.



3. Run It & Follow the Flow

Don't just read the code.
Make it do something.
Try a simple action and watch
what happens.

The running application is often
the best documentation.

You can use:

Console

→ see what's happening

Network

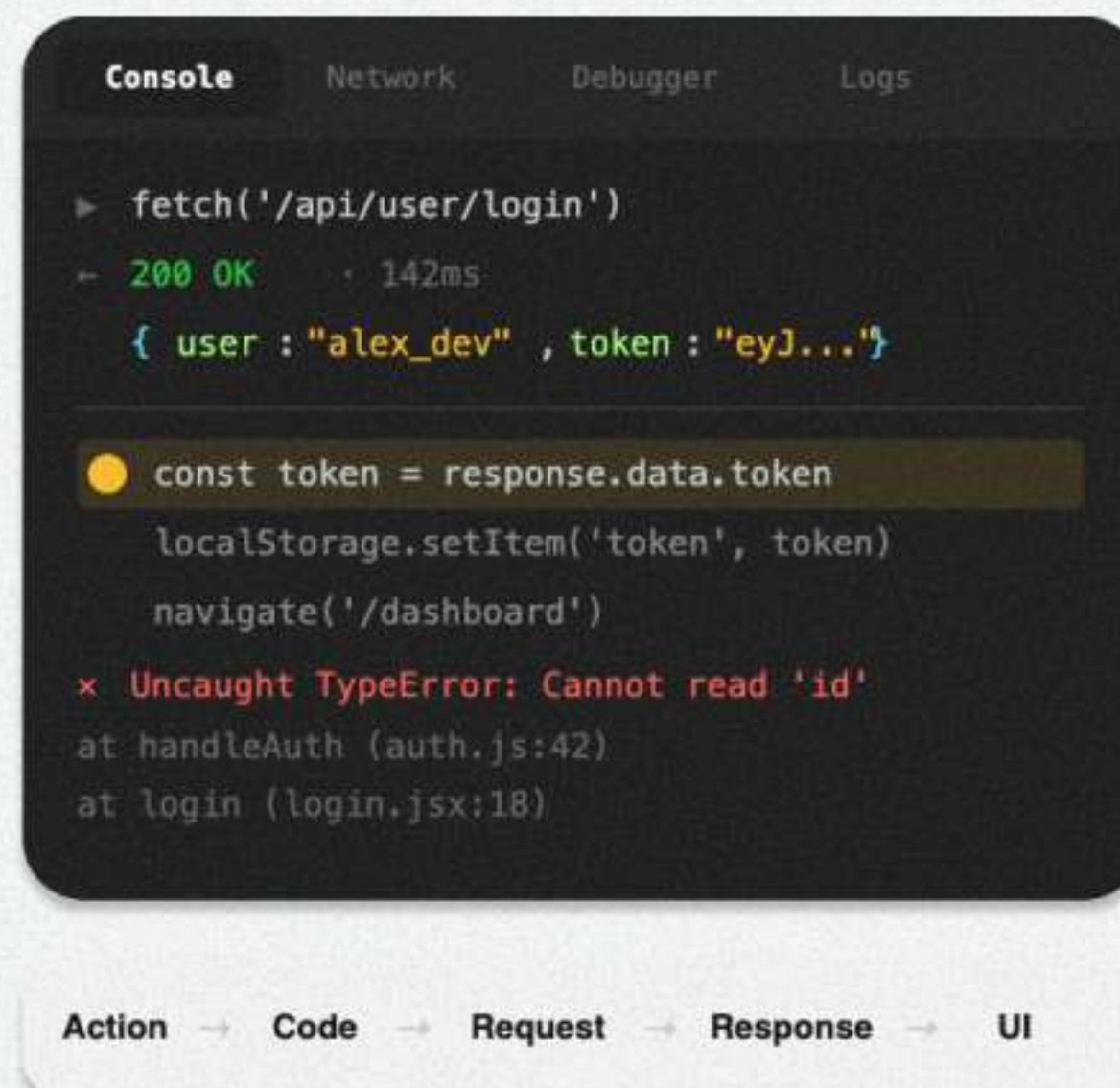
→ see API requests

Debugger

→ pause execution

Logs

→ follow the backend



Where to Look First

Not sure where to start?

Look for these files first:

README

→ **What is this project?**

package.json

→ **What does it use?**

.env.example

→ **What external services/config does it need?**

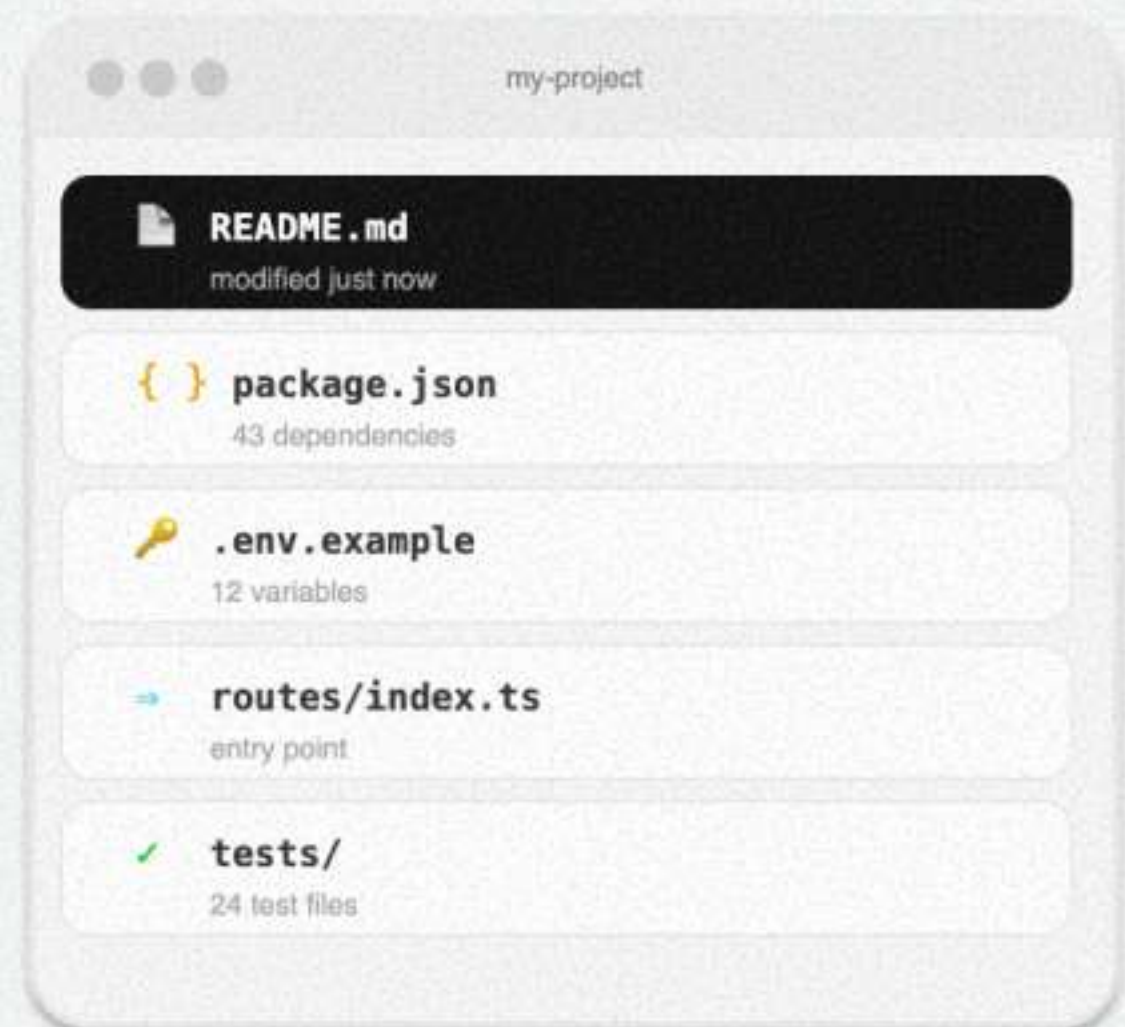
Routes / entry points

→ **Where does the app start?**

Tests

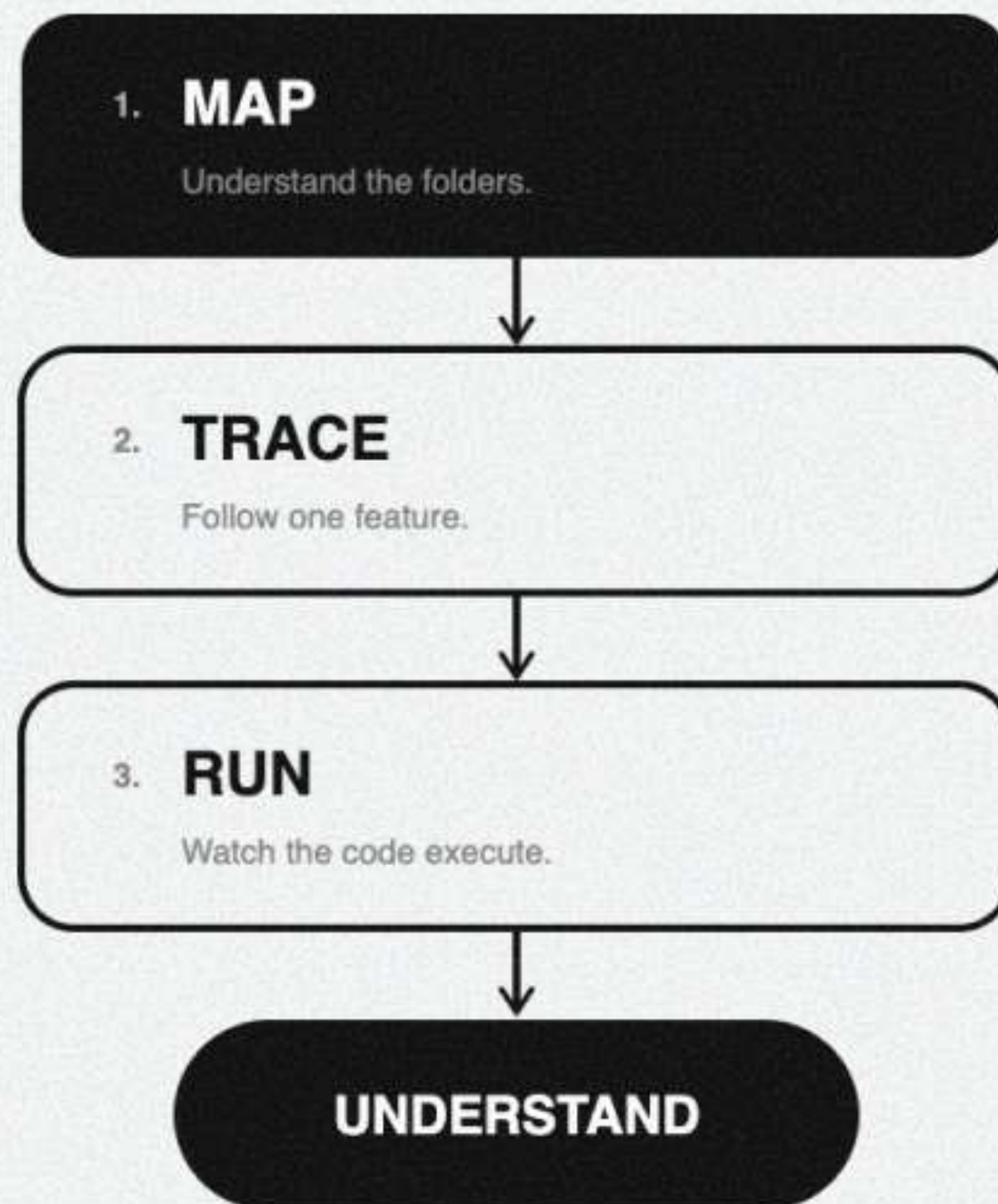
→ **What behavior is expected?**

**Don't read the whole repo.
Find the map first.**



THE CHEAT SHEET

Your codebase-reading cheat sheet



Structure → Feature → Flow

The Bigger Lesson

**Don't understand everything.
Understand one path completely.**

Structure → Feature → Flow

 ***Save this for your next new
repo.***

 ***Comment "CODEBASE" if this
helped you.***